

CollabCode Studio: Real-Time Coding & Visual Collaboration Platform.

A major project report submitted in partial fulfillment of the requirement
for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted by

Major Project Group No. 80

Divyanshu Chander (221030460)

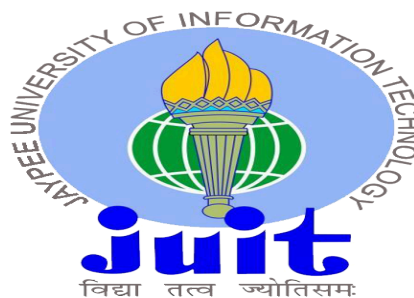
Dhruv Malik (221030423)

Kuldeep Verma (221031057)

Akshat Kapoor (221030296)

Under the guidance & supervision of

Ms. Nitika (Assistant Professor)



**Department of Computer Science & Engineering and
Information Technology**

Jaypee University of Information Technology,

Waknaghat, Solan - 173234 (India)

December 2025

SUPERVISOR'S CERTIFICATE

This is to certify that the major project report entitled '**CollabCode Studio: A real time coding and visual collaboration platform**', submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering, in the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, Waknaghat, is a bona fide project work carried out under my supervision during the period from July 2025 to December 2025. I have personally supervised the research work and confirm that it meets the standards required for submission. The project work has been conducted in accordance with ethical guidelines, and the matter embodied in the report has not been submitted elsewhere for the award of any other degree or diploma.

Supervisor Name: Ms Nitika

Date: 10/12/2025

Designation: Assistant Professor

Department: Department of CSE & IT

CANDIDATES'S DECLARATION

We hereby declare that the work presented in this major project report entitled '**CollabCode Studio: A real time coding and visual collaboration platform**', submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering, in the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, Waknaghat, is an authentic record of our own work carried out during the period from July 2025 to December 2025 under the supervision of Ms Nitika. We further declare that the matter embodied in this report has not been submitted for the award of any other degree or diploma at any other university or institution.

Name: Divyanshu Chander

Roll No.: 221030460

Date: 10/12/2025

Name: Dhruv Malik

Roll No.: 221030423

Date: 10/12/2025

Name: Kuldeep Verma

Roll No.: 221031057

Date: 10/12/2025

Name: Akshat Kapoor

Roll No. 221030296

Date: 10/12/2025

This is to certify that the above statement made by the candidates is true to the best of my knowledge.

Supervisor Name: Ms Nitika

Date: 10/12/2025

Designation: Assistant Professor

Department: Department of CSE & IT

ACKNOWLEDGEMENT

We would like to thank **Jaypee University of Information Technology, Wagnaghat**, for giving us the chance, resources, and educational setting necessary to start and finish our major project, **"CollabCode Studio: Real-Time Coding & Visual Collaboration Platform."**

We want to thank our project supervisor, **Ms. Nitika**, Assistant Professor in the Department of CSE & IT, for her constant support, valuable advice, constructive feedback, and encouragement throughout the project. Her knowledge and guidance were key in shaping the direction and quality of our work.

We would also like to thank the **faculty members** of the **Department of Computer Science & Engineering and Information Technology** for giving us the academic knowledge and technical foundation that helped us finish this project successfully.

We are grateful to our friends and family for their continuous encouragement, tolerance, and support during this journey. Their support has been a continual source of inspiration.

Lastly, we want to thank all the online resources, research papers, frameworks, libraries, and open-source tools that made CollabCode Studio possible. The project would not have reached its current state without them.

With sincere appreciation,

Team CollabCode Studio

Divyanshu Chander (221030460)

Dhruv Malik (221030423)

Kuldeep Verma (221031057)

Akshat Kapoor (221030296)

TABLE OF CONTENTS

LIST OF TABLES	i
LIST OF FIGURES	ii
LIST OF ABBREVIATIONS & NOMENCLATURE	iv
ABSTRACT	vi
1. INTRODUCTION.....	1-6
1.1 Introduction.....	1
1.2 Problem Statement	2
1.3 Objective.....	3
1.4 Significance and motivation of the project report.....	4
1.5 Organization of project report.....	5
2. LITERATURE SURVEY.....	7-16
2.1 Overview of relevant literature.....	7
2.2 Key gaps in the literature.....	15
3. SYSTEM DEVELOPMENT	17-36
3.1 Requirements and Analysis	17
3.2 Project Design and Architecture.....	19
3.3 Implementation.....	23
3.4 Key Challenges.....	35

4. TESTING.....	37-40
4.1 Testing Strategies.....	37
4.2 Test Cases & Outcomes.....	39
5. RESULT & EVALUATION.....	41-43
5.1 Results.....	41
5.2 Comparison with existing solution.....	42
6. CONCLUSIONS & FUTURE SCOPE.....	44-46
6.1 Conclusion.....	44
6.2 Future Scope.....	44
REFERENCES	47-49

LIST OF TABLE

Table no.	Description	Page no.
2.1	Summary of Literature Review	10-16
4.1	Test Cases & Outcomes	36
5.1	Comparative Analysis of Existing Solutions	40

TABLE OF FIGURES

S.no	Title	Page no.
1	Figure 3.1	20
2	Figure 3.2	21
3	Figure 3.3	22
4	Figure 3.4	23
5	Figure 3.5	24
6	Figure 3.6	25
7	Figure 3.7	26
8	Figure 3.8	27
9	Figure 3.9	27
10	Figure 3.10	28
11	Figure 3.11	29
12	Figure 3.12	30
13	Figure 3.13	31
14	Figure 3.14	32

15	Figure 3.15	33
16	Figure 3.16	34
17	Figure 3.17	35
18	Figure 3.18	36

LIST OF ABBREVIATIONS, SYMBOLS OR NOMENCLATURE

ABBREVIATIONS

S.no	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	AWS	Amazon Web Services
4	CRDT	Conflict-free Replicated Data Types
5	CSS	Cascading Style Sheets
6	EC2	Elastic Compute Cloud
7	HTML	HyperText Markup Language
8	HTTP	Hypertext Transfer Protocol
9	JS	JavaScript
10	LLM	Large Language Model
11	MERN	MongoDB, Express.js, React, Node.js
12	OT	Operational Transformation
13	UI	User Interface
14	UX	User Experience
15	LAN	Local Area Network
16	CRUD	Create, Read, Update, Delete

NOMENCLATURE

S.no	Term	Definition
1	Docker	A platform used to containerise applications, ensuring consistency across different environments.
2	Express.js	A backend web application framework for Node.js, used to build the web server and manage API requests.
3	Node.js	A JavaScript runtime environment that executes JavaScript code on the server-side.
4	Piston API	An external API used for executing source code from various programming languages.
5	Pollinations API	An external API that provides access to AI models for features like code suggestions.
6	React	A JavaScript library for building user interfaces, used for the client-side application.
7	Socket.IO	A library that enables real-time, bidirectional, and event-based communication between web clients and servers.
8	WebSocket	A communication protocol that provides a persistent, full-duplex communication channel over a single TCP connection.

ABSTRACT

As remote working and virtual collaboration become the norm, software developers find it difficult to collaborate smoothly. All current tools compel teams to switch between a code editor, a messaging application, and independent visual collaboration tools. The continuous switching not only decelerates the workflow but also causes issues such as miscommunication, version conflicts, and unnecessary delays. Though various collaborative editors do exist, they typically do not have either **uninterrupted real-time synchronisation**, adequate communication features, or the ability to allow for visual brainstorming. Also, very few have intelligent support to assist developers with coding problems and enable them to solve them efficiently.

Our **CollabCode Studio** project takes all these requirements and puts them into one platform. It's a live collaborative space where several users are able to type and edit code at once, view one another's changes in real-time, chat with each other through an integrated chat, and utilise an **interactive whiteboard** for the creation of diagrams or rapid idea sharing. To make the experience even more intelligent, the platform also incorporates **AI-driven features** such as code suggestions and error checking, cutting down on the time teams spend debugging issues.

The system is being implemented with the **MERN** stack (MongoDB, Express, React, Node.js) and Socket.IO for real-time communication. For handling simultaneous edits, we are investigating established algorithms like Operational Transformation (OT) and Conflict-free Replicated Data Types (CRDTs), which are designed to guarantee that everyone sees a consistent version of the code, no matter how many people edit at the same time.

In brief, CollabCode Studio is more than just an online code editor: it unifies live coding, real-time communication, visual collaboration, and AI support into a single unified workspace to make working with others remotely as natural and effective as collaborating face-to-face in the same space.

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION

Nowadays, software is almost never built by just one person. In most cases, especially during college projects or team-based work, many people work together and often from different places. Working in groups ourselves, we realize that collaboration usually becomes messy: we write code in one place, discuss ideas on some chat app, and then switch to another tool if we want to sketch or explain something visually. We constantly move back and forth between these tools, and this affects our concentration and slows us down.

There are various tools that attempt to support collaborative work. Some of them allow several users to write code at the same time, others are mainly used for communication. These tools are helpful, but since they are not connected, problems still occur. Team members get confused during discussions, code versions go out of sync, and the workflow keeps getting disturbed. We also noticed features like drawing designs, managing project files, or getting help with errors usually require extra tools or browser tabs, which makes the entire process more tiring than it should be.

The difficulties indicated a need for creators to create one place where collaboration occurs instead of constantly switching from one tool to another. Creators will be able to write code, collaborate verbally with their teammates, visualise ideas together and receive support where necessary all in the same environment, rather than having to switch applications when writing code. Additionally, the provision of simple AI support functions such as code suggestions and debugging functions from the same platform will save time and ultimately result in an improved development experience for creators due to less time spent debugging code due to having all functions surrounding the project in one location. When there is less movement required in the teamwork environment between team members and development tools, teamwork is made easier and developers are empowered to solve problems easily instead of constantly switching between different tools.

1.2 PROBLEM STATEMENT

With the new technologies that are coming up fast, it has allowed for different ways teams are working together remotely from different places, hence it has now become very important to have a better collaborative software development experience. Even we as students have faced this issue while working together on a common project, that we don't get a unified experience. Also, while looking out for current solutions and ways, they are either expensive or have various shortcomings.

- **Multiple Tool Separation:** During any project development, we have noticed that we kept moving to different tools to write code, talk with our team members, and also share ideas. Even if we somehow managed to do it, it brought so much hassle, our productivity dropped and a simple task would take multiple steps to execute.
- **Limited Integration:** We found that there are many tools and platforms that provide great and solid solutions in a particular domain. Platforms like VS Code Live Share provide us with a shared editing feature, but lack in other domains like built-in communication and visual collaboration. While other tools like Slack or Microsoft Teams did help in communication but they were not of any help for coding.
- **Workflow Inefficiency:** Whenever we used to work on a common project in a group, working together led to increase in complexity and resulted in poor communication, delays in implementation and would make the whole process inefficient.
- **Inadequate Intelligent Support:** Few platforms currently offer **AI-driven capabilities** like real-time error reporting or intelligent code recommendations, forcing developers to turn to third-party plugins or manual debugging.
- **Collaboration Issues:** Switching to multiple tools for a common work resulted in frustration, inconsistent work among team members and it also gave us cross platform challenges due to working in different devices.

So we identified that there is a real issue and a requirement for a common, unique, connected, something which is user friendly in nature ecosystem, that brings these all segregated features together to enable realtime coding, text chatting, visual board brainstorming and support for latest AI tools under a common roof.

1.3 OBJECTIVES

In this project, our aim is to build a realtime collaborative coding platform that makes teamwork easier compared to already available tools. The main goals of this project are:

- A platform will be created to provide a means of facilitating concurrent, collaborative programming via an online programming environment where all members of a development team can edit, compile, build and run a single codebase at the same time, with all code changes viewed immediately by all users.
- Make it possible for users to communicate with each other while working on code instead of switching to a separate messaging app.
- Allow users of the app to use a **shared whiteboard space** to brainstorm and draw diagrams while coding together.
- Add basic **AI support** to help users with code suggestions, error detection, and debugging. It would help us in fast paced and error free development.
- Make sure everyone is working on the same version of the code and avoid conflicts while editing together.
- Keep the platform **easy to use** so that users do not need much training to get started, even a new learner or someone who already has experience can use it.
- Make sure the system can support multiple users at the same time and still perform well. Also when need arises, the project can be scaled to support more number of users.
- Create a collaborative space where people can code and share ideas together without the process feeling forced.

1.4 SIGNIFICANCE AND MOTIVATION FOR THE PROJECT

As we worked on our team project, we found that our largest challenge was not actually writing code but instead switching between tools while we worked remotely. Many modern-day remote developers utilise a variety of independent applications (not only for creating software) to communicate with their peers, visually convey their thoughts and ideas, and generally work on a collaborative basis. As a result, remote developers tend to have to switch from tool to tool all the time, which disrupts their ability to stay focused, thereby reducing the speed at which they complete tasks.

This project's main offering is to try to develop a solution for this problem. Rather than having several different platforms for collaboration, all of the project elements will be combined together into one workspace. The platform will enable companies and students to have better structure and organisation for their collaboration efforts through the minimisation of switching between tools. This will be invaluable for remote teams and students working in collaborative groups because these two groups rely heavily on seamless communication and well defined coordination to successfully complete projects within the designated timeline.

There were several reasons for us to pursue this work, including:

- **Increasing Developer Productivity:** The primary motivation for this project is to assist developers with maintaining concentration when they are coding. In our experience, being forced to switch between multiple applications (like a code editor, chat application, etc.) can not only disrupt a developer's ability to focus but also disrupt the development process badly. With many tools located in a single location, developers will have more time to code, as they will not have to devote so much time to managing their various tools.
- **Reducing the Entry Barrier for Students:** For people who are new to learning programming, setting up a local dev environment might seem really tough. As we know from experience, installation and configuration problems have caused many of our students to not be ready to code themselves. A key focus of this project is providing a simple, zero-setup sandbox that will allow students to jump right in and start coding right away, try new languages and learn from their mentors without having to worry about any technical setup issues.

- **Enhancing Team Collaboration:** Effective communication significantly impacts software project success. As we worked together as teams on multiple software projects, we found that collaboration would flow more easily when everyone shared ideas in the same way comparable to how brainstorming occurs in face-to-face meetings. This software project is intended to create an environment where developers would be able to collaborate within their coding environment. By combining your coding work with both verbal and written interaction, teams are able to keep track of all communications, drawings, and code in one location.
- **Encouraging Innovation through AI:** Numerous organizations are using modern AI-enhanced code writing/classifiers as an incentive to increase their product development abilities by introducing products that support their existing products' ability to develop more quickly, provide assistance on solving coding problems, shorten the time between programming errors and correcting them, and to provide an overall higher level of quality for all software.

1.5 ORGANISATION OF PROJECT REPORT

This project is divided into six chapters which are given below:

1. Chapter 1: Introduction – introduces the project by explaining its background, the problem being addressed, the objectives, and the motivation behind the work. It also briefly explains how the rest of the report is organised.
 - 1.1 Introduction
 - 1.2 Problem statement
 - 1.3 Objectives
 - 1.4 Significance and Motivation of the Project Work
 - 1.5 Organization of Project Report
2. Chapter 2: Literature Review – This chapter reviews previous work related to the project and highlights areas where existing solutions fall short. It helps explain what

this project aims to achieve and shows where improvements are still needed in the current industry practices.

2.1 Overview of Relevant Literature

2.2 Key Gaps in the Literature

3. Chapter 3: 3.1-System Development – This chapter explains how the system was planned and developed. It discusses the main requirements, the overall design, the system architecture, and the tools and technologies used to build the project.

3.1 Requirements and Analysis

3.2 Project Design and Workspace

3.3 Implementation

3.4 Key Challenges

4. Chapter 4: Testing

4.1 Testing Strategies

4.2 Test Cases & Outcome

5. Chapter 5: Result & Evaluation

5.1 Result

5.1.1 Functional Completion

5.1.2 System Performance

5.2 Comparison with Existing Solution

5.2.1 Conclusion of Comparison

6. Chapter 6: Conclusion and Future Scope

6.1 Conclusion

6.2 Future Scope

CHAPTER 2: LITERATURE REVIEW

2.1 OVERVIEW OF RELEVANT LITERATURE

This study examined various aspects of the creation of the CollabCode Studio application, including real-time collaboration, leveraging AI technologies for software development and utilizing web technology to create a scalable, robust application. Much of our understanding came from the literature that is available on these topics.

2.1.1 REAL-TIME COLLABORATION AND DATA CONSISTENCY

A primary challenge in collaborative editors is maintaining data consistency across multiple users. The literature points to two main approaches: **Operational Transformation (OT)** and **Conflict-free Replicated Data Types (CRDTs)**. While OT has been a traditional solution, recent studies have demonstrated that CRDTs are favorable due to their inherent conflict-resolution capabilities and suitability for decentralized systems. The work of I. David and E. Syriani [1] has successfully applied CRDTs on real-time collaborative modeling and demonstrated how concurrent edits will merge smoothly with no complex transformation logic. Moreover, the research by P. S. Almeida [2] gives an exhaustive review of several approaches towards CRDTs, which further justifies choosing them as a state-of-the-art, robust solution to handle shared state in collaborative applications.

2.1.2 AI IN SOFTWARE ENGINEERING

The integration of Artificial Intelligence is slowly but surely changing the software development landscape. Our review affirms that AI indeed has the potential to substantially improve developer productivity. R. Ferdiana's work [5] points to positive effects of using AI in programming tasks, while the detailed taxonomy provided by C. Treude and M. A. Gerosa [4] on how developers interact with AIs provides several insights helpful for the design of intuitive features for human-AI collaboration. A critical aspect of integrating AI is evaluating its performance. The survey by **Liguo Chen et al.** [3] on evaluating Large Language Models (LLMs) for code generation offers a crucial framework for selecting and validating the effectiveness of the AI models that will power our platform's code suggestion and error-checking features.

2.1.3 ENABLING TECHNOLOGIES AND ARCHITECTURE

The technological background of the platform is based on the latest web technologies. WebSockets are recommended as the standard of the industry in the context of real-time communication. The study was conducted by A. Gote [6] and M. B. Khan et al. [8] proves the idea that WebSockets offer a low-latency, two-way communication channel that is persistent and required to provide an uninterrupted collaborative experience.

In the case of the application architecture, the literature supports the MERN (MongoDB, Express.js, React, Node.js) stack as a potent option when creating dynamic web applications. The impact of Node.js in developing efficient and scalable applications on the server side is described by G. Jadhav and F. Gonsalves [9], and the performance advantages of the React tool on the development of interactive user interface are examined by I. Yusron and A. Wibowo [12]. Lastly, in order to be scalable and reliable, the deployment on Amazon Web Services (AWS) is considered. The articles by R. Saini and R. Behl [10] and B. Bachina [11] presents some useful instructions on how to create and deploy MERN apps on AWS, which influence our plan of deployment.

Table 2.1: Summary of Literature Survey

Sr No.	Author / Title / Journal	Key Techniques / Focus	Advantages	Limitations
1.	P. S. Almeida - Approaches to Conflict-free Replicated Data Types - (ACM), 2024	A survey that compares the two main CRDT approaches: operation-based and state-based.	It provides a foundational understanding of CRDTs and their trade-offs, guiding the choice of a suitable approach to ensure eventual consistency.	Being a survey, it reviews existing work and does not introduce new algorithms & metrics.
2.	I. David & E. Syriani - Real-time collaborative multi-level modeling by conflict-free replicated data types - (Software and Systems Modeling), 2022	Proposes a framework using custom CRDTs for real-time collaboration in model-driven engineering (MDE).	Successfully enables scalable and robust real-time collaboration, ensuring eventual consistency automatically.	Can have high memory overhead as data structures grow.

3.	M. Kotsovoulou & V. Stefanou - Student Perceptions on the Effectiveness of Collaborative Problem-based Learning Using Online Pair Programming Tools - (Journal Article), 2021	A qualitative study using a phenomenographic research approach to analyse student interviews about their experience with online pair programming tools.	The study found that students perceive these tools as highly effective for problem-solving, increasing participation, and enhancing engagement and knowledge sharing.	This is a qualitative study focused on student perceptions and does not provide quantitative data on how these tools impact specific metrics like completion time or code quality.
4.	C. Treude & M. A. Gerosa - How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering - (arXiv preprint), 2025	Proposes a taxonomy to classify the different interaction types between developers and AI tools in software engineering.	Identifies and defines eleven distinct types of human-AI interaction, providing a clear framework for designing and evaluating AI-powered developer tools.	Conceptual paper; lacks empirical data to validate its impact.
5.	R. Ferdiana - The Impact of Artificial Intelligence on Programmer Productivity - (Conference Paper), 2024	Review and analysis of how AI tools, particularly code assistants like GitHub Copilot, are changing the software	Argues that AI boosts productivity by automating tasks, shifting the developer's role to focus on	Conceptual overview; lacks new empirical data or quantitative metrics.

		development process and the role of the programmer.	high-level system design.	
6.	G. Jadhav & F. Gonsalves - Role of Node.js in Modern Web Application Development - (IRJET), 2020	Review of the Node.js runtime environment, focusing on its architecture which uses an event-driven, non-blocking I/O model to handle concurrent requests efficiently.	Shows Node.js is significantly faster and more scalable for concurrent tasks, handling 949.12 requests/sec compared to Apache's 237.19. This efficiency is critical for real-time applications.	The provided performance test is based on a specific use case, not a comprehensive benchmark across various workloads.
7.	A. Gote - Real-time Interactivity in Hybrid Applications With Web Sockets - (IRJMETS), 2024	Review of WebSockets as a technology for enabling real-time, bidirectional communication. It explores three specific use cases in modern hybrid applications.	WebSockets are a powerful and pivotal technology for creating the seamless, real-time interactivity that modern web and mobile users expect.	It does not introduce a new protocol or provide a quantitative performance comparison against other real-time technologies like long-polling.

8.	M. C. da Silva et al. - A Visual Tool for Supporting Collaborative Code Quality - (CSCWD Conference), 2019	Describes a visual tool that integrates various software metrics (like complexity and coupling) to create visualizations of code quality, helping teams collaborate on improvements.	The tool successfully provides a shared visual understanding of code quality issues, which facilitates more effective team discussions and collaborative decision-making on refactoring efforts.	Presents the tool and its conceptual framework; it does not include a large-scale empirical study with industry developers to quantitatively measure its impact on productivity or code quality.
9.	R. Saini & R. Behl - An Introduction to AWS - EC2 (Elastic Compute Cloud) - (ICRMAT Conference), 2020	An introduction and review of Amazon Web Services' EC2, explaining its role in cloud computing and how it provides scalable, on-demand virtual server instances.	Highlights EC2's key benefits: flexibility, cost-effectiveness, and scalability, which allow organizations to easily acquire and manage computing resources	This is a high-level introductory paper. It describes the service and its benefits but does not include a technical deep-dive.

10.	B. Bachina - Architecting and Deploying MERN Stack Applications on AWS ECS - (Journal of Scientific and Engineering Research), 2022	A comprehensive guide detailing the step-by-step process of deploying a MERN stack application on AWS ECS, using Docker for containerisation.	Practical roadmap for developers to achieve a scalable, reliable, and easily managed deployment of a MERN application in a cloud-native environment.	Does not provide novel performance metrics or an analysis against alternative deployment platforms.
11.	I. Yusron & A. Wibowo - A Performance Analyst Comparison of ReactJS and AngularJS in the Front-End Website – (IJSait), 2020	A comparative performance analysis where two identical applications were built using ReactJS and AngularJS, then subjected to load testing using Apache JMeter.	The results demonstrated that ReactJS is significantly more efficient, showing faster response times and the ability to handle a higher number of concurrent requests compared to AngularJS.	The study is based on a single application and compares ReactJS to AngularJS (the older version), not the more modern Angular framework.

12.	Liguo Chen et al. - A Survey on Evaluating Large Language Models in Code Generation – (Survey Paper), 2024	A comprehensive survey that reviews and categorises the benchmarks, metrics (e.g., correctness, efficiency), and tasks used to evaluate the performance of LLMs in code generation.	Provides a crucial framework for evaluating and selecting an appropriate LLM for the project's AI features. It helps ensure the AI support is effective and reliable.	As a survey, it reviews existing work and does not introduce a novel evaluation method or a new LLM.
13.	M.B Khan et al. - Design and Development of Real-time Code Editor for Collaborative Programming - (Int. J. of Scientific Research in Computer Science and Engineering), 2023	Describes the design and implementation of a web-based, real-time collaborative code editor using WebSocket Technology for communication	It demonstrates the use of WebSockets to achieve the real-time synchronisation needed for a collaborative editor.	Does not provide a comparative analysis of other key synchronization methods like CRDTs or Operational Transformation (OT)
14.	Fiala, Yee-King & Grierson Collaborative Coding Interfaces on the Web (CodeCircle)(ICLI, 2016)	Describes CodeCircle, a unified web platform combining IDE, live coding, and pastebin features with real-time collaboration (using	This paper directly addresses the unified platform gap, detailing the integration of editing, assets, and collaboration	Collaboration is primarily text-based (editor); lacks explicit integration of a shared visual brainstorming canvas or

		ShareJS/OT) and a secure asset system.	features into one system..	integrated AI features..
15.	S.W Lee & Essl Communication, Control, and State Sharing in Networked Collaborative Live Coding (UrMus)(NIME, 2014)	Introduces a system using individual and shared namespaces to manage state conflicts, integrated live chat, and a "Live Variable View" for state awareness.	The use of namespaces and live variable views provides solutions for state-level conflict management and enhancing coder communication/awareness in a centralized system.	Focuses on Live Music/Audio coding (Lua/UrMus); the solutions are not generalized to the broader general software development or visual collaboration needs.

2.2 KEY GAPS IN THE LITERATURE

While going through the existing literature related to collaborative development tools, we found that many technologies are well studied but there are still some clear gaps that are not properly addressed. These gaps are closely related to the problems we faced while working on many projects requiring team efforts and they helped shape the direction of this work.

- Lack of a Common Unified Platform:** Literature gap is in the absence of a truly unified platform that we are working on. Most research papers focus on only one part of collaboration at a time. Some discuss real-time code editing, while others focus on idea sharing through video calls or messaging tools. Very few studies look

at building a single system where coding, communication, and visual collaboration are all available together in one workspace.

- **Limited Focus on a Holistic User Experience:** There is a lack of focus on the overall user experience and working on the user level. There are many papers that look into developing technical features like synchronization algorithms, but areas of decreasing switching tools and making collaboration feel as close to real experience are not explored in detail.
- **Scarcity of Integrated AI in Collaborative Editors:** There is research that talks about how AI is used in programming and how it helps developers. Most of this work focuses on individual programmers and their productivity. However very little research shows how AI is actually used by real software teams working together in collaborative environments.
- **Insufficient Research on Combined Feature Sets:** There are very little studies that exist that really look to assess our motive behind the project and the different benefits it would give. We believe that this project will fill this blank space by creating and conducting a study on the platform which has all the benefits we are trying to achieve.

CHAPTER 3: SYSTEM DEVELOPMENT

3.1 REQUIREMENTS AND ANALYSIS

The aim of this study is to create an all-in-one development and collaboration environment that allows software developers to build applications more effectively. During the analysis of the system requirements we gave careful attention to creating a single platform that integrates both the tools needed for development and collaboration. Our goal is to enable software developers to collaborate and work more efficiently together in realtime and ultimately increase their productivity. The primary types of requirements are summarized below.

3.1.1 FUNCTIONAL REQUIREMENTS

Functional requirements tell us about all the required features and behaviour of the system based on users expectations. In this some of the important ones are discussed below:

- **Session Management:** Users will create new workspace sessions that are called rooms. They can also join existing rooms made by someone else. The state of each room will be maintained by session management which will include a list of users that are currently present, their code files in the room, etc.
- **Realtime Collaborative Code Editor:** As the code editor space is the primary component of this project it will allow users in the room to make changes in their code. If anybody is currently making any change, it would be reflected on the editor space highlighting the user name and it will be updated to everyones system.
- **Integrated Real-Time Chat:** Every workspace room will have a chat feature that would help room members to chat with each other without leaving the workspace.
- **Shared Visual Drawing Board:** All the users in the room will have access to a shared drawing board which would help them brainstorm ideas and create their project design, UML diagrams, etc.
- **AI Powered Assistance:** The project will offer users access to an **AI assistance** which can suggest programming code, complete missing sections of the program, and detect errors in the process of producing the program.

3.1.2 NON-FUNCTIONAL REQUIREMENTS (how well the system must perform)

Non functional requirements describe how well the system should work. They focus on performance reliability and usability which directly affect the user experience.

- **Performance:** The system should have lower latency in the shared features that are updated in realtime. User inputs, keystrokes, messages and drawings should be updated fastly to feel responsive and natural
- **Scalability:** The backend architecture of the project must be able to handle a large number of users and sessions at a given time without causing performance issues.
- **Availability and Reliability:** The app should be error free and be highly available and reliable and it should be ensured that there is as minimal downtime as possible.
- **Usability:** The UI of the application must be easy to work with, understandable, minimal and should provide better user experience. The user should easily be able to work on it from the very first use.

3.1.3 TECHNICAL ANALYSIS

Our group analysis shows that this project's overall performance depends highly on realtime sync of services. The challenge would be to decrease network latency in all this and manage conflicts. Keeping in mind all this we explored and selected the relevant web technology stack for the benefit of our project:

- **Frontend:** For frontend development our team is comfortable with using ReactJS library which helps in making single page applications. We will also use Vite which is a tool that enables fast development experience and helps in making dynamic and responsive UI. Many libraries like React Router DOM will help us in client side routing. Tailwind CSS will be used due to experience and ease of use for styling the application.
- **Backend:** The backend logic will be handled by Node.js. The API requests for creating rooms, etc will be handled by it. The environment variables used during development will be managed using dotenv library provided in the Node.js development environment.

- **Real-Time Communication:** Real time communication is handled by a dedicated Socket.IO server. It manages persistent two way WebSocket connections. These connections support all real time features such as the shared code editor chat and the drawing board.
- **Synchronisation Algorithm:** The synchronisation algorithm will use CRDTs to handle how text edits are merged between users. These updates will be sent using the Socket.IO webSocket connection. Hence this will keep the data consistent for all the users without needing a special control mechanism to resolve conflicts.
- **Code Editor Component:** The code editor will provide code editing features such as syntax highlighting and autocompletion.
- **Deployment:** The application will be packaged using Docker so that it behaves the same during development testing and deployment. This helps avoid environment related issues. The system will be deployed on a cloud service such as AWS EC2 at later stages. This setup supports scalability and helps maintain good availability and performance.

3.2 PROJECT DESIGN AND ARCHITECTURE

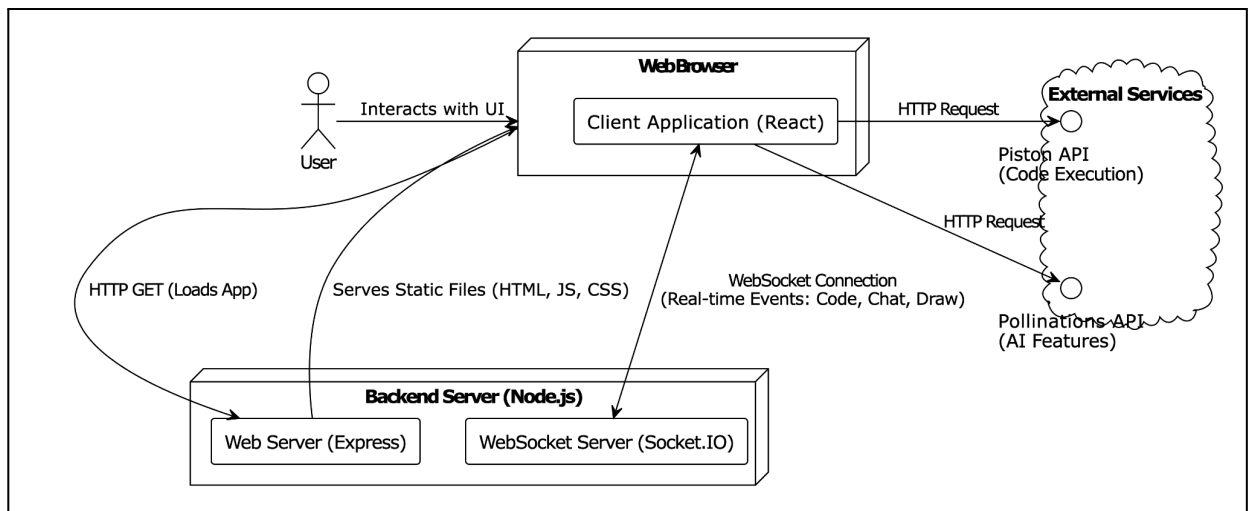


Fig. 3.1 High Level System Architecture

This diagram shows how the Client Application, Backend Server and External Services interact with each other with the help of using both WebSockets and HTTP and also directly to third-party services through their APIs.

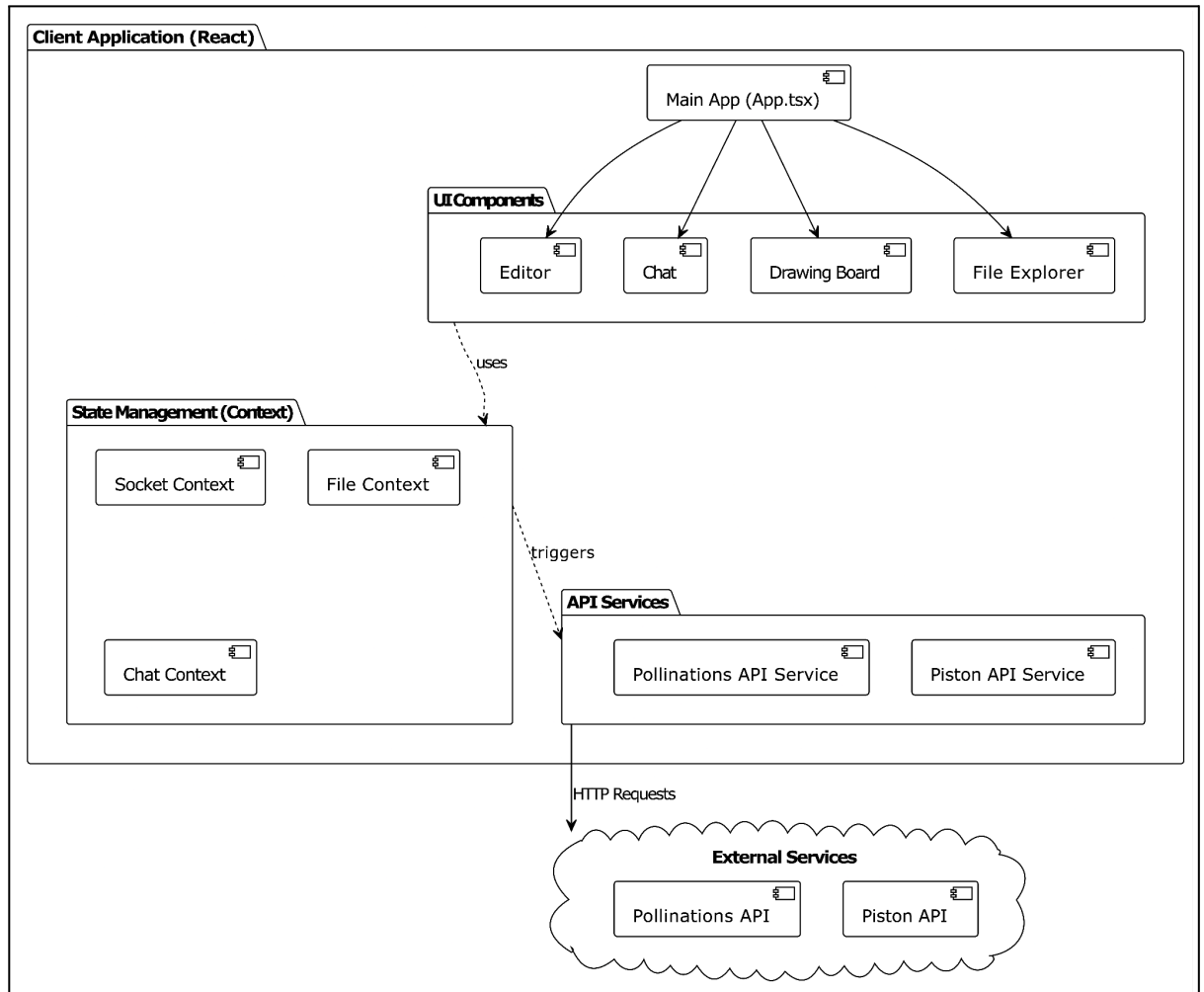


Fig. 3.2 Client Application Diagram

It describes the communication between various components such as UI Editor, Chat, Drawing Board and File Explorer in the application and also the management of the states of these components. It is also shown that the external services are used to our advantage for various features of our project.

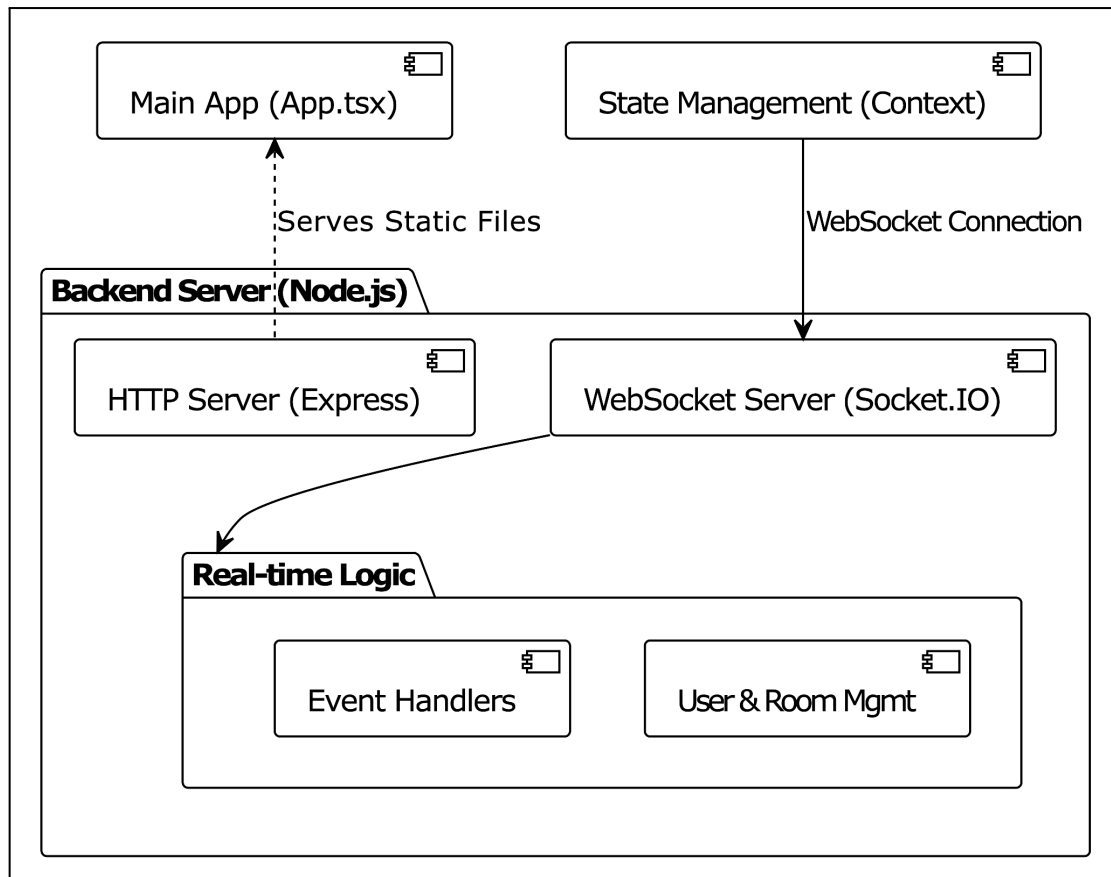


Fig. 3.3 Backend Server Diagram

The Express HTTP server provides the means for serving up static files. Socket.io provides users with real time communication through its webSocket server. Layout and logic layers allow for the management of users as well as event driven operations. Together these allow people to work efficiently while keeping their data in sync with one another.

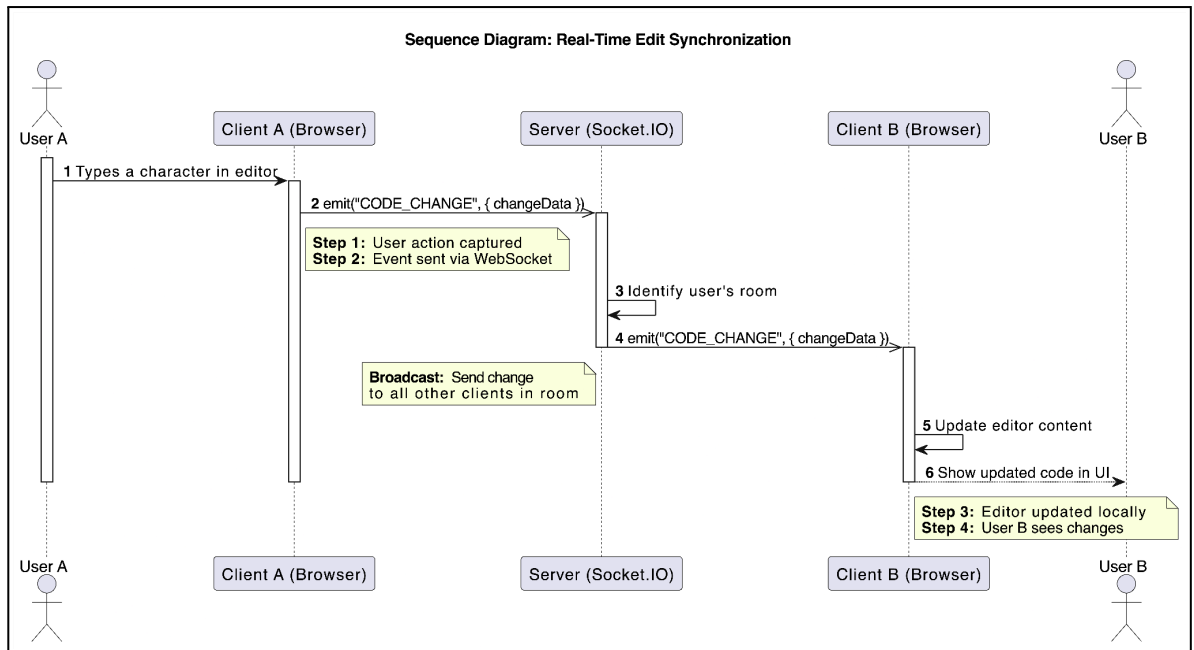


Fig. 3.4 Sequence Diagram of Realtime Edit Synchronisation

It shows how a change or edit in the code editor is transmitted and updated to all the other users in the room. For this room ID given by user A is used to help in realtime changes.

3.3 IMPLEMENTATION

3.3.1 PROJECT DIRECTORY STRUCTURE

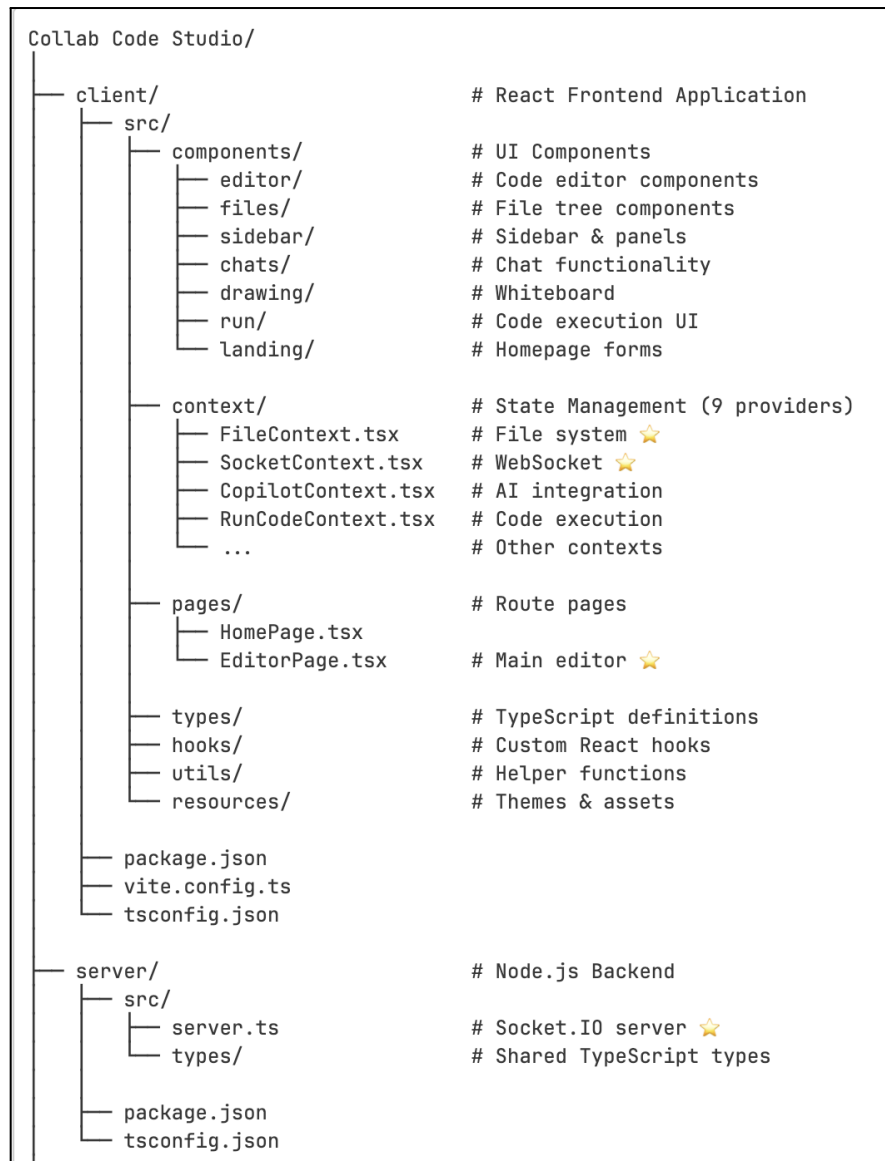


Fig. 3.5 Project Directory Structure

3.3.2 SERVER INITIALIZATION & CONFIGURATION

```
const app = express();
app.use(express.json());
app.use(cors());
app.use(express.static(path.join(__dirname, "public")));

const server = http.createServer(app);
const io = new Server(server, {
  cors: {
    origin: "*", // Allow all origins for collaboration
  },
  maxHttpBufferSize: 1e8, // 100MB buffer for large file transfers
  pingTimeout: 60000,      // 60s timeout to handle slow connections
});

let userSocketMap: User[] = []; // Global state for active users
```

Fig. 3.6 Initialisation & Configuration of ExpressJS server

The snippet shows the initialisation of the express server with socket.IO and configuration for cross origin requests and large file handling. The userSocketMap helps in maintaining realtime user state across all rooms in our application.

3.3.3 HELPER FUNCTIONS FOR ROOM & USER MANAGEMENT

```
// Function to get all users in a room
function getUsersInRoom(roomId: string): User[] {
  return userSocketMap.filter((user) => user.roomId === roomId);
}

// Function to get room id by socket id
function getRoomId(socketId: SocketId): string | null {
  const roomId = userSocketMap.find(
    (user) => user.socketId === socketId
  )?.roomId;

  if (!roomId) {
    console.error("Room ID is undefined for socket ID:", socketId);
    return null;
  }
  return roomId;
}

function getUserBySocketId(socketId: SocketId): User | null {
  const user = userSocketMap.find((user) => user.socketId === socketId);
  if (!user) {
    console.error("User not found for socket ID:", socketId);
    return null;
  }
  return user;
}
```

Fig. 3.7 Room & User Management Helper Functions

A utility function which allows for efficient lookup of rooms and users, to route socket events to the correct user(s) of that particular room to keep data uniform and synchronized across all users who occupy that room, is illustrated in this snippet.

3.3.4 USER JOIN REQUEST & DISCONNECTION

```
socket.on(SocketEvent.JOIN_REQUEST, ({ roomId, username }) => {
  // Check if username exists in the room
  const isUsernameExist = getUsersInRoom(roomId).filter(
    (u) => u.username === username
  );

  if (isUsernameExist.length > 0) {
    io.to(socket.id).emit(SocketEvent.USERNAME_EXISTS); // Reject duplicate
    return;
  }

  // Create new user object with initial state
  const user = {
    username,
    roomId,
    status: USER_CONNECTION_STATUS.ONLINE,
    cursorPosition: 0,
    typing: false,
    socketId: socket.id,
    currentFile: null,
  };

  userSocketMap.push(user); // Add to global state
  socket.join(roomId); // Join Socket.IO room
  socket.broadcast.to(roomId).emit(SocketEvent.USER_JOINED, { user }); // Notify others
  const users = getUsersInRoom(roomId);
  io.to(socket.id).emit(SocketEvent.JOIN_ACCEPTED, { user, users }); // Send user list
});
```

Fig. 3.8 User Join Request

The code handles new user connections by validating if the username is unique or not. After that it broadcasts join events to existing users and sends a complete list of existing users to the new joiner to ensure all clients stay in sync when new users join the room.

```
socket.on("disconnecting", () => {
  const user = getUserBySocketId(socket.id);
  if (!user) return;

  const roomId = user.roomId;
  socket.broadcast.to(roomId).emit(SocketEvent.USER_DISCONNECTED, { user });
  userSocketMap = userSocketMap.filter((u) => u.socketId !== socket.id); // Cleanup
  socket.leave(roomId);
});
```

Fig. 3.9 User Disconnection & Cleanup

The code handles user disconnections by notifying room members that someone has left the room and after that cleaning up server state. Uses the disconnecting event which is run before leaving rooms to access roomId.

3.3.5 FILE STRUCTURE SYNCHRONISATION

```
socket.on(  
  SocketEvent.SYNC_FILE_STRUCTURE,  
  ({ fileStructure, openFiles, activeFile, socketId }) => {  
    io.to(socketId).emit(SocketEvent.SYNC_FILE_STRUCTURE, {  
      fileStructure,    // Complete file tree  
      openFiles,        // Currently opened tabs  
      activeFile,       // Active editor file  
    });  
  })  
);
```

Fig. 3.10 File Structure Synchronisation

The code helps in file structure sync across users. When a new user joins then an existing user sends their complete file system directly to the new user via the server.

3.3.6 REAL TIME CRUD OPERATIONS

```
// File Creation
socket.on(SocketEvent.FILE_CREATED, ({ parentId, newFile }) => {
  const roomId = getRoomId(socket.id);
  if (!roomId) return;
  socket.broadcast
    .to(roomId)
    .emit(SocketEvent.FILE_CREATED, { parentId, newFile });
});

// File Content Update
socket.on(SocketEvent.FILE_UPDATED, ({ fileId, newContent }) => {
  const roomId = getRoomId(socket.id);
  if (!roomId) return;
  socket.broadcast.to(roomId).emit(SocketEvent.FILE_UPDATED, {
    fileId,
    newContent,
  });
});

// File Rename
socket.on(SocketEvent.FILE_RENAMED, ({ fileId, newName }) => {
  const roomId = getRoomId(socket.id);
  if (!roomId) return;
  socket.broadcast.to(roomId).emit(SocketEvent.FILE_RENAMED, {
    fileId,
    newName,
  });
});

// File Deletion
socket.on(SocketEvent.FILE_DELETED, ({ fileId }) => {
  const roomId = getRoomId(socket.id);
  if (!roomId) return;
  socket.broadcast.to(roomId).emit(SocketEvent.FILE_DELETED, { fileId });
});

// Similar pattern for DIRECTORY operations (CREATE, UPDATE, RENAME, DELETE)
```

Fig. 3.11 File CRUD Operation

The code broadcasts patterns for file operations. Each operation:

1. Retrieves the room ID on the sender socket side
2. After that it tells it to all users except the sender (using socket.broadcast)
3. And finally maintains consistency across all the users in the room.

The server here only acts like a message relay helping in communication without storing file state anywhere.

3.3.7 LIVE CURSOR & TYPING INDICATOR

```
socket.on(SocketEvent.TYPING_START, ({ cursorPosition }) => {
  // Update user state in global map
  userSocketMap = userSocketMap.map((user) => {
    if (user.socketId === socket.id) {
      return { ...user, typing: true, cursorPosition };
    }
    return user;
  });

  const user = getUserBySocketId(socket.id);
  if (!user) return;
  const roomId = user.roomId;

  // Broadcast typing state to others
  socket.broadcast.to(roomId).emit(SocketEvent.TYPING_START, { user });
});

socket.on(SocketEvent.TYPING_PAUSE, () => {
  userSocketMap = userSocketMap.map((user) => {
    if (user.socketId === socket.id) {
      return { ...user, typing: false };
    }
    return user;
  });

  const user = getUserBySocketId(socket.id);
  if (!user) return;
  const roomId = user.roomId;
  socket.broadcast.to(roomId).emit(SocketEvent.TYPING_PAUSE, { user });
});
```

Fig. 3.12 Cursor & Typing Indicator

It tracks the realtime changes done by the user whenever they type. Helps in showing the cursor position of the current changes.

- TYPING_START: Updates the cursor position and the typing status of the user
- TYPING_PAUSE: Clears the typing indicator when the user stops typing

Something like "User X is typing" is shown in the editor.

3.3.8 DRAWING BOARD SYNCHRONISATION

```
socket.on(SocketEvent.REQUEST_DRAWING, () => {
  const roomId = getRoomId(socket.id);
  if (!roomId) return;
  socket.broadcast
    .to(roomId)
    .emit(SocketEvent.REQUEST_DRAWING, { socketId: socket.id });
});

socket.on(SocketEvent.SYNC_DRAWING, ({ drawingData, socketId }) => {
  socket.broadcast
    .to(socketId)
    .emit(SocketEvent.SYNC_DRAWING, { drawingData });
});

socket.on(SocketEvent.DRAWING_UPDATE, ({ snapshot }) => {
  const roomId = getRoomId(socket.id);
  if (!roomId) return;
  socket.broadcast.to(roomId).emit(SocketEvent.DRAWING_UPDATE, {
    snapshot,
  });
});
```

Fig. 3.13 Drawing Board Synchronisation

It helps in synchronizing changes of drawing board across all the users

1. REQUEST_DRAWING: New user requests the current state the drawing is in
2. SYNC_DRAWING: Existing user sends their canvas data to rest of the users
3. DRAWING_UPDATE: Realtime stroke update is done across all the other users.

3.3.9 NETWORK IP DISCOVERY

```
function getLocalIP(): string {
  const interfaces = os.networkInterfaces();
  for (const name of Object.keys(interfaces)) {
    for (const iface of interfaces[name]!) {
      if (iface.family === "IPv4" && !iface.internal) {
        return iface.address;
      }
    }
  }
  return "localhost";
}

server.listen(Number(PORT), "0.0.0.0", () => {
  const localIP = getLocalIP();
  console.log(`Server running on:`);
  console.log(`  Local:   http://localhost:${PORT}`);
  console.log(`  Network: http://${localIP}:${PORT}`); // LAN access
});
```

Fig. 3.14 Network IP Discovery for Development

It helps in providing the local network IP address which enables the feature to work on a local area network (LAN) facilitating development.

3.3.10 LOGIN SCREEN IMPLEMENTATION



Fig. 3.15 Implementation of login screen

This is our current implementation of a simple login screen, we have provided the option to add room id and your username. If the room ID is already made a user will be added to it or else a new room will be created.

3.3.11 EDITOR INTERFACE

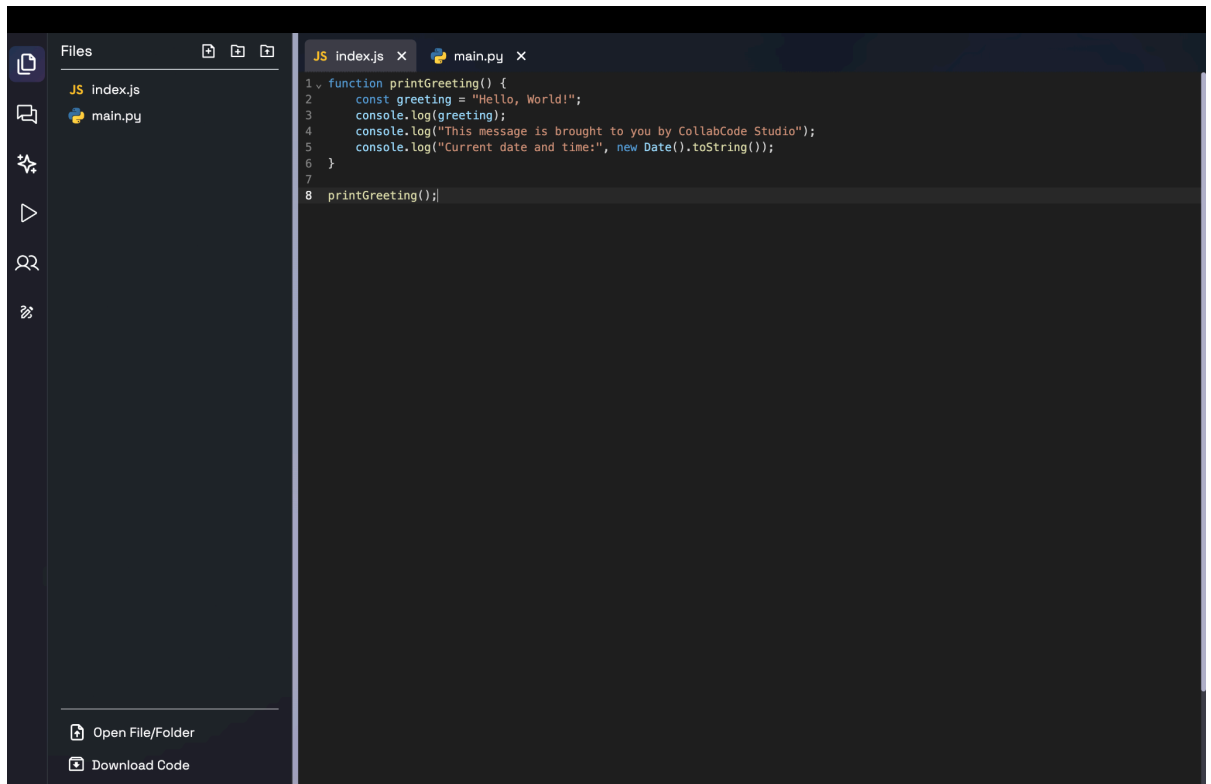


Fig. 3.16 Code Editor Interface

The interface of the code editor shows two main parts. The right side of the page shows the editing window with the ability to write code and the left side of the page shows the options menu with features available like chat function, AI code generation, white board collaboration, etc.

3.3.12 WHITEBOARD IMPLEMENTATION

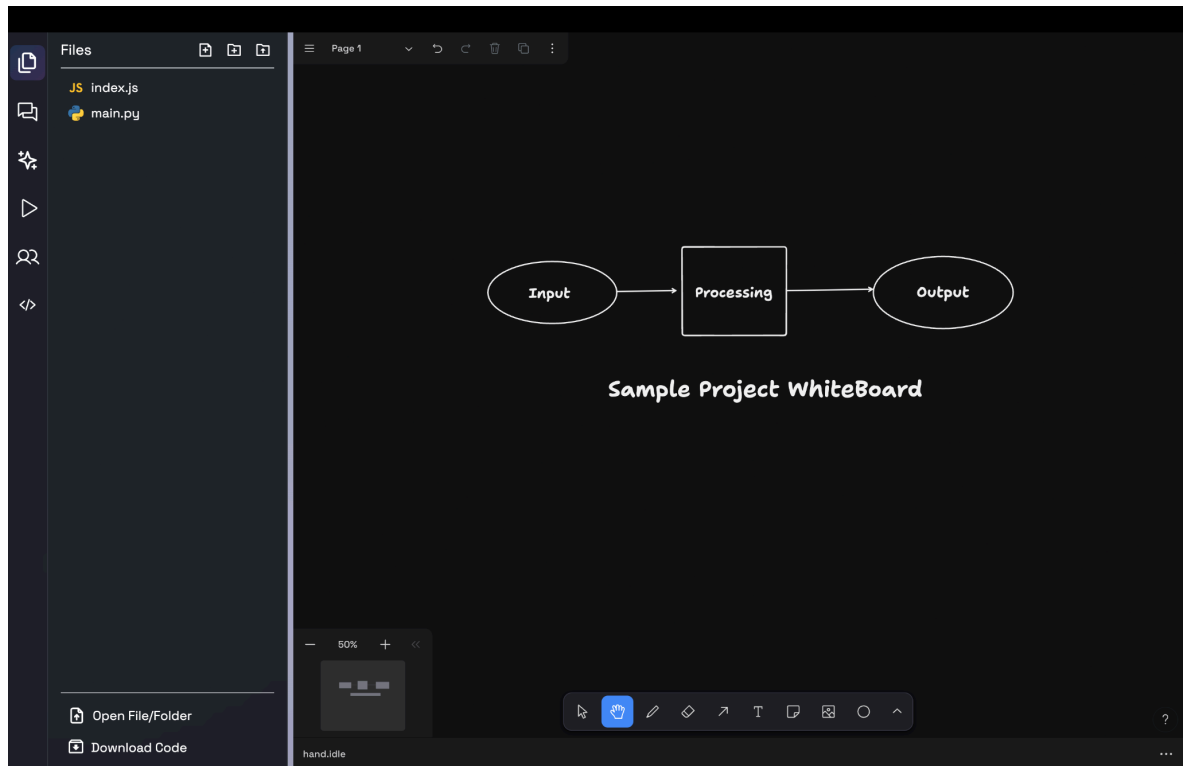


Fig. 3.17 Whiteboard Interface

We implemented a sample whiteboard space where users can make different designs, ideate projects, create UML diagrams, etc all while collaborating together in synchronisation. This whiteboard also provides various functionality of multiple tools, design download option and many more other important features.

3.3.13 REALTIME CHAT INTERFACE

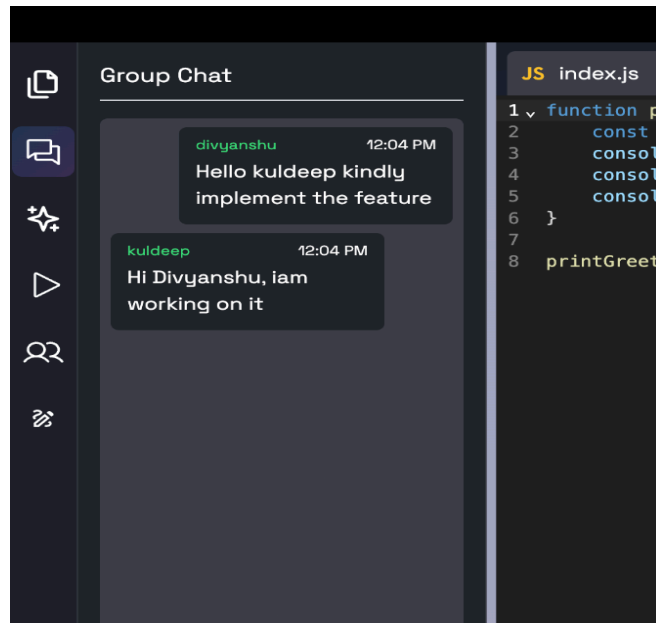


Fig. 3.18 Realtime Chat Interface

A real time chat interface is also implemented by us which helps us to talk and discuss and share messages in real time to our team members in the project.

3.4 KEY CHALLENGES

- 1. File Structure Sync Across Users :** Whenever a new user joined the active session or room their workspace did not get updated and was showing blank even when other users had made changes and added newer documents. This problem was resolved by making the old users share their complete file structure whenever someone new joined the room.
- 2. Duplicate File Name Conflicts :** When different users in our room tried to create a file with the same name or name which was already present the file got replaced and lost. To solve this problem we implemented an automatic feature that would add numbers at the end of filename to not cause data loss. For example: if an “index.js” file is already present, the newer file would add 1 at the end like “index(1).js”.

- 3. Showing Where Other Users Are Typing :** We wanted users to see where other collaborators were working inside the code editor. This helps everyone understand who is editing which part of the code. To do this we created a small extension for the CodeMirror editor. It shows a tooltip with the user name above the cursor. This makes it easier to track activity during collaboration.
- 4. Socket memory Performance Problems:** There was slowness of the application after it was used extensively. This occurred due to the fact that socket event listeners were accumulated and they were not cleared. We addressed the same by making sure that all the event listeners are removed on closing or updating of components. This assists in maintaining the memory usage even in case of extended cooperation.
- 5. Mismatch of data between the frontend and the Backend:** At most times the front end and the backend required different data structures of the same information. This led to incompatibility in communication. We resolved to this by sharing type definition files which are used both on the client and the server. It will guarantee that both sides always adhere to the same data structure.
- 6. Whiteboard API :** Building a whiteboard from scratch is difficult and time consuming. Because of this we chose to use tldraw. It supports multiplayer features uses very little system resources and works well with Socket.IO. This choice helped us save several weeks of development effort instead of building everything ourselves.

CHAPTER 4: TESTING

This chapter presents and discusses the testing methodology followed by us, the test cases executed, and the outcomes of the tests that together validate the working and reliability of the platform.

4.1 TESTING STRATEGIES

To make sure real world collaboration works correctly the system needed detailed testing. CollabCode Studio includes many real time features such as the code editor chat whiteboard and external API integration. Because all these parts work together the platform required thorough testing to confirm performance reliability and user experience.

Test Methods Included:

- Manually validating user flow.
- Conducting real-time tests of the platform with multiple users connected.
- Validating compatibility of CollabCode Studio across different browser types.
- Evaluating platform performance under high load.
- validating the integration of CollabCode Studio with other external services.

All categories of testing were developed specific to the functional and non-functional requirements of the platform.

4.1.1 Manual Testing

The performance of the user interface, navigation routes, and expected results for all primary elements were initially validated through some form of manual testing. Manual testing of the various features included File Operations, Code Editor, Room Builder, Team Services and Whiteboard. The purpose of this manual testing was to confirm that each and every user-facing component was functioning as intended and were not exhibiting unexpected behaviour.

4.1.2 Realtime Collaboration Testing

CollabCode Studio is focused on synchronous collaboration, so it conducted tests with 3 to 5 people actively collaborating from shared workspaces. Test results confirmed the

functionality of many aspects of multi-user collaboration including simultaneous editing, mouse tracking, typing indicators, real-time whiteboard synchronization and conflict-free updates across different types of networks.

4.1.3 Cross Browser & Device Testing

The application underwent extensive quality assurance testing for uniform display and functionality across many of the most popular browsers: Chrome, Firefox, Edge, and Safari. Once QA testing was completed on desktop systems, additional testing was performed on mobile devices such as smartphones and tablets to evaluate the application's ability to support multiple operating systems (cross-platform compatibility), as well as its responsiveness to various screen sizes (responsive design), and how it would feel to users who interact with it using touch screens (touch interaction performance).

4.1.4 Integration Testing

The results of integration testing confirmed how the front-end interacts with the back-end, Socket.IO server and other services like the Piston API (for executing multi-language code) and the Pollinations API (for generating AI code). Through these tests we were able to verify the proper response from our APIs, our ability to handle errors, and how to insert the generated output back into the editor.

4.2 TEST CASES & OUTCOME

Table 4.1: Test Cases & Outcomes

Test Case No.	Feature Tested	Expected Output	Status
01	Create Room	Room is created and user joins successfully	Pass
02	Join Room	User connects to an existing room	Pass
03	File Creation	New File is Created	Pass
04	Rename & Delete File	File is renamed/deleted	Pass
05	Duplicate File Name	System appends (1), (2), etc.	Pass
06	Code Editing	Code updates appear in real-time for all users.	Pass
07	Cursor Sync	Collaborate Cursor with the user	Pass
08	Switch to WhiteBoard	Whiteboard loads and syncs for all users.	Pass
09	Real Time Drawing	Drawing Syncs Across all clients	Pass
10	AI Code Suggestion	AI generated code appears in editor	Pass
11	Code Execution (Piston API)	Output/errors displayed correctly	Pass
12	Browser Compatibility	Chrome, Firefox, Edge, Safari supported	Pass
13	User Disconnect	User leaves without affecting session state	Pass
14	Invalid Room Join	Error Message Displayed	Pass
15	Cross Device LAN functioning	Sync between different devices on the same network	Pass

- Total test cases executed & passed = 15
- Including file creation, file editing, code synchronization, execution, and generation of code using artificial intelligence (AI).

According to the results of all testing to date, it appears that the project is ready to proceed with further application development due to meeting the applicable criteria for continued

development of the application. Testing has verified that the core features of instant editing of code, the ability to collaborate in real time with other users, the ability to send and receive messages in chat, the ability to use a common white board to write ideas down collaboratively, and the ability to interface with various external APIs have all performed correctly across a variety of devices and operating systems as well as different types of networks.

CHAPTER 5: RESULTS & EVALUATION

This chapter presents the results of development and testing at Collab Code Studio, showing how it works on real time collaboration, system responsiveness, user experience, and dependability. The evaluation also considers how well the platform achieves its objectives and compares it to other solutions for collaborative coding. It focuses on the measurement of functional completeness and collaborative behavior in multiuser environments.

5.1 RESULTS

5.1.1 FUNCTIONAL COMPLETION

All development objectives of Collab Code Studio are met effectively. The following features were implemented and verified:

1. Real-Time Collaborative Editor

- Many people can edit the same file at the same time.
- Socket.IO keeps edits in sync in real time with no inconsistency.
- All connected users receive reflections of cursor positions and typing indicators.

2. Project Structure Synchronisation

- It sends the entire file and folder structure to any new user who joins a session.
- Conflicts of duplicate file names are resolved automatically.

3. Integrated Live Chat

- Seamless conversation in a room, with no third-party apps necessary.

4. Interactive Whiteboard

- Multi-user real-time drawing with stroke-level synchronization.
- Smooth loading of the existing canvas state when new users join.

5. AI Powered Features

- Snippets are generated and code recommended through API calls.
- Smooth reinsertion of generated output back into the active editor.

In general, the system implemented provides a complete and stable collaboration environment, meeting the main objectives of this project.

5.1.2 SYSTEM PERFORMANCE

The key performance observations are:

1. Realtime Latency

- Editor updates spread to every client depending on network speed.
- It has a very smooth cursor and typing indicators even when there are four to six users at once.

2. Whiteboard Synchronisation

- Updates at the stroke level are transmitted immediately and with very little latency.
- Loading canvas state is very accurate and fast for new users.

3. Server Stability

- The socket server also managed to keep track of multiple users over extended sessions.

4. Cross Browser Results

- Testing of each functionality was successful on: Safari, Firefox, Chrome.

5. Multi Device Behaviour

- The same LAN network showed consistency of collaborative actions between laptops, tablets, and mobile devices.

5.2 COMPARISON WITH EXISTING SOLUTION

Collab Code Studio was compared with the state-of-the-art real-time collaboration platforms like Google Colab, Replit Multiplayer, and VS Code Live Share. All these tools have the facility for collaborative editing; however, none of them unifies all the required features into a single working environment.

Table 5.1: Comparative Analysis with Existing solutions

Feature/Criteria	VS Code Live Share	Replit Multiplayer	Google Colab	CollabCode Studio
Real-Time Code Editing	Yes	Yes	Limited	Yes(via Socket.IO)
Built-in Chat	No	Limited	Yes	Yes
Shared Visual Board	No	No	No	Yes (Real-Time, Multi-user)
AI Code Suggestion	Yes(via extension)	Yes	Yes	Yes (Integrated)
Multi-Language Support	Yes	Yes	Python Only	Yes (via Piston API)
File Structure Sync	Limited	Yes	No	Yes
Setup Effort	High (requires VS Code)	Medium	Low	Very Low (Browser-based)
One-Stop Collaboration	Partial	Partial	Partial	Fully Unified Workspace

5.2.1 CONCLUSION OF COMPARISON

As such, the comparison research unequivocally illustrates that no platform in use today provides, in one setting, all the features necessary for a comprehensive, real-time collaborative coding experience.

What makes Collab Code Studio unique is its provision of:

- One platform for conversation, scripting, art, and AI support.
- Synchronization of all features in real time.
- Easy onboarding with access limited to browsers.
- An atmosphere that is conducive to learning and collaboration.
- A multimodal cooperative experience emulating real-life face-to-face collaboration.

CHAPTER 6: CONCLUSIONS AND FUTURE SCOPE

6.1 CONCLUSION

Collab Code Studio is an effective attempt at providing a cohesive real-time code collaboration workspace. This eliminates the frustration of using multiple applications to code, chat and share ideas. By combining a live code editor, collaborative whiteboard, live chat alongside realtime file system integration and AI assistance inside the same roof, this project makes your work so productive and space is never an issue because we all know how teams cooperate these days in real development.

It is developed with Node.js, Express, React, and Socket.IO provide for low latency communication, interoperability between various users and devices. Additional features include cursor tracking, typing indicators, directory integration, automatic conflict resolution of file name conflicts, and smooth onboarding, all of which add up to even better collaboration. Testing proves the reliability of realtime functionality, browser compatibility, and AI. It proved that the platform runs well while handling a high amount of concurrency in users.

In other words, the Collab Code Studio can be viewed as a powerful example of an integrated collaboration tool. The project meets its objectives but at the same time creates the real possibility for enhancements, including persistent storage, full IDE like features, and much further AI integration in the future. The successful development and testing of the system confirm its potential for further development as a professional or educational collaboration platform.

6.2 FUTURE SCOPE

Although the current version of Collab Code Studio already features all key aspects intended for MVP, there are many points at which the platform can further be augmented and extended to increase its strength, scalability, and usefulness for wider applicability.

1. Advanced Realtime Synchronisation

- Operational Transformation or CRDT based algorithms may be supported for better handling of the conflict resolution process in large scale collaborations.
- The offline editing should be enabled, but it should automatically be synchronized when one is reconnected to the network.

2. Development Support based on Artificial Intelligence

- Include autocompletion, which is driven by real-time AI, such as GitHub Copilot.
- Increase the level of context sensitivity in order to get better code completion and error explanation.
- Automate generation of tests, code refactoring recommendations and AI code inspection.

3. Better Customisation

- Include alternatives of customisation and options that can change the theme, behaviour, inclination etc. of the editor.

4. Better Packaging and File system.

- Installation and management of external dependencies Support the same as npm or pypi.
- Either have sessions that are independent of each other, in which case there are no version conflicts between collaborators.
- Auto sync the installed packages to make sure that the same environment is shared among all the users.

5. Persistent Project Storage

- One should save projects in the cloud so that the project session can be maintained even when the users are not in the office.
- Include project history, version management and collaborative checkpoint.

6. Better Room Authentication and User Management

- Authentication-based entry to the room.
- Different privileges, roles, and hierarchy in one room.
- One user can have access to several rooms.

7. Scalability and Deployment Improvements

- Shift to microservices architecture for better load distribution.

- Implement load balancers and distributed socket management that will support higher levels of concurrency.
- The solution includes monitoring and analytics dashboards that offer insights into performance.

8. Cross Platform Support

- Build specialized desktop apps with Electron and mobile apps with React Native.
- Enable push notifications, offline mode, and session management consistency.

The foregoing improvements could turn Collab Code Studio from a prototype resulting from academic interest into a fully fledged Collaborative Development Environment that would easily be acceptable for usage both in classrooms, remote teams and hackathons, and professional settings. In any case, the system's modular architecture provides an easy path for forthcoming updates to be smoothly integrated with the production version.

REFERENCES

- [1] I. David and E. Syriani, "Real-time collaborative multi-level modeling by conflict-free replicated data types," *Software and Systems Modeling*, 2022. [Online]. Available: <https://experts.mcmaster.ca/display/publication3281463>
- [2] P. S. Almeida, "Approaches to Conflict-free Replicated Data Types," *arXiv preprint arXiv:2310.18220*, 2023. [Online]. Available: <https://arxiv.org/html/2310.18220v2>
- [3] L. Chen, et al., "A Survey on Evaluating Large Language Models in Code Generation," *arXiv preprint arXiv:2408.16498*, Aug. 2024. [Online]. Available: https://www.researchgate.net/publication/383529947_A_Survey_on_Evaluating_Large_Language_Models_in_Code_Generation_Tasks
- [4] C. Treude and M. A. Gerosa, "How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering," *arXiv preprint arXiv:2501.08774*, 2025. [Online]. Available: <https://arxiv.org/pdf/2501.08774>
- [5] R. Ferdiana, "The Impact of Artificial Intelligence on Programmer Productivity," in *Conference Paper*, 2024. [Online]. Available: https://www.researchgate.net/publication/394745464_The_Impact_of_Artificial_Intelligence_on_Programmer_Jobs_Threat_or_Opportunity
- [6] A. Gote, "Real-time Interactivity in Hybrid Applications With Web Sockets," *International Research Journal of Modernization in Engineering Technology and Science (IRJMETs)*, 2024. [Online]. Available: https://www.researchgate.net/publication/378871302_Real-time_Interactivity_in_Hybrid_Applications_With_Web_Sockets
- [7] M. C. da Silva, et al., "A Visual Tool for Supporting Collaborative Code Quality," in *Proc. 23rd International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, 2019. [Online]. Available: https://www.researchgate.net/publication/333516254_A_Visual_Tool_for_Supporting_Collaborative_Code_Quality
- [8] M. B. Khan, et al., "Design and Development of Real-time Code Editor for Collaborative Programming," *International Journal of Scientific Research in Computer Science and Engineering*, vol. 11, no. 6, pp. 19-26, Dec. 2023. [Online]. Available: https://isroset.org/journal/IJSRCSE/full_paper_view.php?paper_id=2533
- [9] G. Jadhav and F. Gonsalves, "Role of Node.js in Modern Web Application Development," *International Research Journal of Engineering and Technology (IRJET)*, vol. 7, no. 6, 2020. [Online]. Available: <https://www.irjet.net/archives/V7/i6/IRJET-V7I61149.pdf>
- [10] R. Saini and R. Behl, "An Introduction to AWS - EC2 (Elastic Compute Cloud)," in *Proc. International Conference on Recent Mechanical and Automation Technology (ICRMAT)*, 2020. [Online]. Available: https://www.researchgate.net/publication/348638365_An_Introduction_to_AWS-EC2_Elastic_Compute_Cloud

- [11] B. Bachina, "Architecting and Deploying MERN Stack Applications on AWS ECS," *Journal of Scientific and Engineering Research*, vol. 9, no. 2, pp. 123-142, 2022. [Online]. Available: <https://jsaer.com/download/vol-9-iss-2-2022/JSAER2022-9-2-123-142.pdf>
- [12] I. Yusron and A. Wibowo, "A Performance Analyst Comparison of ReactJS and AngularJS in the Front-End Website," *International Journal of Science and Applied Information Technology (IJSait)*, 2020. [Online]. Available: <https://www.semanticscholar.org/paper/Research-and-Analysis-of-the-Front-end-Frameworks-Xing-Huang/7a252edb5c9f003aee8400c109b778a350ee1411>
- [13] M. Kotsovoulou and V. Stefanou, "Student Perceptions on the Effectiveness of Collaborative Problem-based Learning Using Online Pair Programming Tools," *WSEAS Transactions on Proof*, vol. 1, pp. 15-22, 2021. [Online]. Available: <https://wseas.com/journals/articles.php?id=5463>
- [14] J. Fiala, M. Yee-King, and M. Grierson, "Collaborative coding interfaces on the Web," in Proc. International Conference on Live Coding (ICLI), 2016. [Online]. Available: <https://www.google.com/search?q=https://iclc.livecodenetwork.org/2016/papers/fiala-et-al-2016.pdf>
- [15] S. W. Lee and G. Essl, "Communication, Control, and State Sharing in Networked Collaborative Live Coding," in Proc. International Conference on New Interfaces for Musical Expression (NIME), 2014. [Online]. Available: https://www.nime.org/proceedings/2014/nime2014_554.pdf
- [16] OpenAI, *ChatGPT: GPT-5 Model*, OpenAI, San Francisco, CA, USA. [Online]. Available: <https://chat.openai.com>
- [17] Microsoft, *Visual Studio Code (VS Code)*, Microsoft Corporation, Redmond, WA, USA. [Online]. Available: <https://code.visualstudio.com>
- [18] Google, *Gemini: Google DeepMind's Multimodal AI*, Google LLC, Mountain View, CA, USA. [Online]. Available: <https://deepmind.google/technologies/gemini>
- [19] Node.js Foundation, *Node.js JavaScript Runtime Environment*, OpenJS Foundation. [Online]. Available: <https://nodejs.org>
- [20] Meta Platforms, Inc., *React: A JavaScript Library for Building User Interfaces*, Meta, Menlo Park, CA, USA. [Online]. Available: <https://react.dev>
- [21] Socket.IO, *Socket.IO: Real-Time Bidirectional Event-Based Communication*, OpenJS Foundation. [Online]. Available: <https://socket.io>

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

• Bibliography

Match Groups

- **72 Not Cited or Quoted 11%**
Matches with neither in-text citation nor quotation marks
- **2 Missing Quotations 1%**
Matches that are still very similar to source material
- **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- **1 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 11% ● Internet sources
- 5% ● Publications
- 7% ● Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



JAYPEE UNIVERSITY OF INFORMATION TECHNOLOGY, WAKNAGHAT
PLAGIARISM VERIFICATION REPORT

Date: 11/12/25

Type of Document (Tick): ☒ PhD Thesis ☐ M.Tech/M.Sc. Dissertation ☐ B.Tech./BCA/BBA Report

Name: Divyanshu Chander Department: CSE Enrolment No 221030460

ORCID ID. SCOPUS ID.

Contact No. 7018612535 E-mail. 221030460@juitplan.in

Name of the Supervisor: Ms. Nitika

Title of the Thesis/Dissertation/Project Report/Paper (In Capital letters): COLLABCODE
STUDIO: REAL-TIME CODING & VISUAL COLLABORATION
PLATFORM

UNDERTAKING

I undertake that I am aware of the plagiarism related norms/ regulations, if I found guilty of any plagiarism and copyright violations in the above thesis/report even after award of degree, the University reserves the rights to withdraw/revoke my degree/report. Kindly allow me to avail Plagiarism verification report for the document mentioned above.

- Total No. of Pages = 63
- Total No. of Preliminary pages = 12
- Total No. of pages accommodate bibliography/references = 2

Divyanshu Chander
(Signature of Student)

FOR DEPARTMENT USE

We have checked the thesis/report as per norms and found Similarity Index : 12 (%) and AI Writing: 0% [] or *% [✓]. (Please [✓] any one % as per generated report). Therefore, we are forwarding the complete thesis/report for final plagiarism check. The plagiarism verification report may be handed over to the candidate.

16/12/25
(Signature of Guide/Supervisor)

Signature of HOD

FOR LRC USE

The above document was scanned for plagiarism check. The outcome of the same is reported below:

Document Received Date	Excluded	Similarity Index (%)		Title, Abstract & Chapters Details	
	All Preliminary Pages	Overall Similarity		Word Counts	
Report Generated Date	Bibliography / References	AI Writing		Character Counts	
	Images/Quotes	0%		Page counts	
	14 Words String	*%		File Size	

Checked by
Name & Signature

Librarian

Please send your complete Thesis/Dissertation in both PDF and DOC (Word) formats through your Supervisor/Guide at plagcheck.juit@gmail.com

(Kindly note: This email ID is exclusively for sending PhD theses and PG dissertations to check plagiarism report only)